小さなビルドツール

Golem プロジェクト

目次

第 1 章　依存関係を読む …… 5
1.1　依存のグラフ …… 5
1.1.1　循環する依存 …… 5
1.2　依存を書くファイル …… 6
1.2.1　依存を自動で見つける …… 6
1.2.2　読み込みのオプション …… 6
第 2 章　再ビルドを判定する …… 9
2.1　時刻で比べる …… 9
2.2　指紋で比べる …… 9
2.2.1　二つの方法の比べ方 … 9
2.2.2　ハッシュの記録 …… 10
2.2.3　用語 …… 10
第 3 章　同時に動かす …… 13
3.1　動かす順 …… 13
3.2　出力を揃える …… 13
3.2.1　混ざる出力 …… 13
3.2.2　出力をためる …… 14
3.3　失敗を扱う …… 14
3.3.1　止めるか続けるか …… 14
3.3.2　書きかけのファイル … 14
3.4　作ったものを使い回す …… 15
3.4.1　キャッシュ …… 15
3.4.2　使い回せる条件 …… 15
3.5　まとめ …… 15
参考文献 …… 15
索引 …… 17

第 1 章　依存関係を読む

1.1　依存のグラフ

ビルドツールの仕事は、変わったものだけを作り直すことである。そのためには、どのファイルがどのファイルから作られるかを知っていなければならない。この関係を依存関係と呼び、矢印でつないだ図にしたものをグラフと呼ぶ。図 1-1 は、四つのファイルの依存関係を表している。

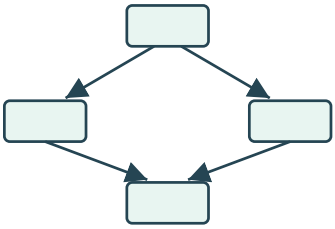


図 1.1　四つのファイルの依存関係。矢印は「作るのに使う」を表す。

一番上のファイルを作るには、その下の二つが要り、その二つを作るには、さらに一番下のファイルが要る。グラフを下から順にたどれば、どの順に作ればよいかが分かる。この順を求める手順は、トポロジカルソートとして知られている[1]。

1.1.1　循環する依存

グラフに循環があると、どこから作ればよいかが決まらない。A が B を要り、B が A を要るなら、どちらも先に作れないからである。ビルドツールは、循環を見つけたらエラーにして止まるのがよい。黙ってどちらかを先に作ると、作るたびに結果が変わってしまう。循環の見つけ方は、深さ優先の探索で、たどっている途中の頂点にもう一度出会うかどうかを調べるのが簡単である。探索の途中の頂点を覚えておくには、頂点ごとに「未訪問」「訪問中」「訪問済み」の三つの状態を持たせればよい。「訪問中」の頂点に出会ったら、そこから今の頂点までが循環である。

> 作り直すべきものだけを作り直し、作り直さなくてよいものに

*1 グラフの頂点を、辺の向きに矛盾しない順に並べる手順。[1] は、ビルドツールをこの順の決め方と再ビルドの判定の組み合わせとして整理している。

> は手を触れない。これがビルドツールの約束のすべてである。
>
> ある古いビルドツールの手引きから

1.2　依存を書くファイル

依存関係は、一行に一つのターゲットと、そのターゲットが要るファイルを並べたテキストのファイルに書くことにする。次の例は、 deps.txt という名前のファイルの中身である。

```
app.o app.c app.h config.h
util.o util.c util.h
app app.o util.o
```

一行目は、app.o が app.c と app.h と config.h から作られることを表す。三行目は、実行ファイルの app が二つのオブジェクトファイルから作られることを表す。ファイルの大きさは問わないが、この例のファイルは 2 KiB ほどの依存を書いても、読み込みに一秒もかからない。

1.2.1　依存を自動で見つける

依存関係を手で書くのは、間違いのもとである。ヘッダーファイルを一つ足したのに deps.txt を直し忘れると、そのヘッダーを変えても作り直されない。そこで、多くのビルドツールは、コンパイラーに依存関係を書き出させる。C のコンパイラーの多くは、ソースファイルが読み込んだヘッダーファイルの一覧を、make の形式で書き出すオプションを持っている。この一覧を deps.txt に足せば、手で書く依存は、ソースファイルとターゲットの対応だけになる。ただし、書き出された一覧は、前回のコンパイルのときのものである。ヘッダーファイルを新しく読み込むようにソースを変えたときは、その変更そのものがソースファイルの変更なので、ソースファイルの時刻かハッシュで作り直しが決まる。作り直したときに、新しい一覧が書き出される。この順のおかげで、前回の一覧を使っても、見落としは起きない。

1.2.2　読み込みのオプション

ツールのオプションは、次の表のとおりである。この表は、Emet の関数が TSV のファイルを読んで作っている。

ツールが受け付けるオプション。

オプション	意味
-j N	同時に動かす数
-k	失敗しても続ける
-n	動かさずに表示する
-B	すべて作り直す
-C DIR	DIR に移ってから動く

オプションの名前は、makeの慣習にならった[*2]。慣れた名前を使えば、利用者は説明を読まずに使い始められる。

ビルドツールを自分で作るのは、車輪の再発明に見えるかもしれない。しかし、小さなツールを一つ作ってみると、既存のツールが何を判断しているのかがよく分かる。どのファイルを作り直すかの判定、作る順の決め方、同時に動かす数の制御は、どのビルドツールにも共通する三つの柱である。この本では、その三つを順に作りながら、それぞれの判断の中身を確かめていく。作るのは、シェルスクリプトで百行に満たない小さなツールである。それでも、三つの柱の考え方は、大きなビルドツールと変わらない。大きなツールが持つ機能の多くは、この三つの柱の上に、速さや使いやすさのための工夫を積み重ねたものにすぎない。たとえば、リモートのキャッシュは再ビルドの判定を、分散ビルドは同時に動かす数の制御を、それぞれ広げたものと見ることができる。三つの柱が分かっていれば、新しいツールに出会っても、それがどの柱をどう広げたものかを考えれば、使い方の見当がつく。この見当のつけ方こそ、小さなツールを作る一番の収穫である。もちろん、作ったツールをそのまま仕事に使う必要はない。仕事では、多くの人が使い、多くの不具合が直されてきたツールを使うほうが安全である。小さなツールは、読んで理解するためのものであり、大きなツールの設定を読むときの手がかり

*2 GNU make の同じ名前のオプション（[2]）と同じ意味にしている。

になる。手元で動く小さな見本があれば、大きなツールの説明書の一文一文が、どの判断の話をしているのかを確かめながら読める。

第 2 章　再ビルドを判定する

2.1　時刻で比べる

一番簡単な判定は、ファイルの更新の時刻を比べることである。ターゲットより新しいファイルが一つでもあれば、ターゲットを作り直す。make はこの方法を使っている。リスト 2-1 は、この判定をシェルスクリプトで書いたものである。

リスト 2.1　時刻で判定するビルドスクリプト

```
set -eu
while read -r target deps; do
  for dep in $deps; do
    if [ ! -e "$target" ] || [ "$dep" -nt "$target" ]; the
      n
      echo "rebuild $target because $dep is newer t
        han the target or the target does not exist yet
        "
      sh "rules/$target.sh"
      break
    fi
  done
done <deps.txt
```

時刻の比較には、シェルの -nt の演算子を使っている。ファイルの数が多くても、時刻を読むだけなので速い。ただし、この方法には弱点がある。ファイルの中身が変わらなくても、時刻が変われば作り直してしまう。逆に、時刻が戻ると、変わったファイルを見落とす。チェックアウトのたびに時刻が変わるバージョン管理のシステムでは、不要な作り直しが頻繁に起きる。時刻の精度もシステムによって違い、一秒の単位でしか記録しないファイルシステムでは、一秒の間に二回書き換えたファイルの変更を見落とす。

2.2　指紋（しもん）で比べる

もう一つの方法は、ファイルの中身から計算したハッシュ値を比べることである。ハッシュ値は、ファイルの中身の指紋のようなもので、中身が少しでも変われば別の値になる。前回のビルドのときのハッシュ値を記録しておき、今のハッシュ値と比べれば、本当に中身が変わったファイルだけが分かる。

2.2.1　二つの方法の比べ方

二つの方法の費用を、千二百のファイルを持つプロジェクトで測った。結果は次の表のとおりである。この表は CSV のファイルから読んでいる。

処理	ファイル数	秒
依存関係の読み込み	1200	0.4
時刻の比較	1200	0.1
ハッシュの計算	1200	2.8
コンパイル	85	41.0
リンク	1	3.2

ハッシュの計算は、時刻の比較の二十倍以上の時間がかかる。それでも、コンパイルの時間に比べれば小さい。不要なコンパイルを一つ減らせば、ハッシュの計算の費用は取り戻せる。

二つの方法の性質を、表にまとめておく。

性質	時刻	ハッシュ
速さ	速い	遅い
時刻の変化	作り直す	作り直さない
時刻の巻き戻し	見落とす	見落とさない
記録の要否	要らない	要る
精度の制限	ある	ない
中身の比較	しない	する
大きなファイル	速い	遅い
生成したファイル	時刻次第	中身次第

性質	時刻	ハッシュ
ネットワーク越しのファイル	時計のずれに弱い	ずれに強い

2.2.2　ハッシュの記録

ハッシュで比べるには、前回のハッシュ値をどこかに記録しておかなければならない。この本のツールは、ターゲットごとに一行を書いたテキストのファイルを、ビルドのディレクトリに置く。一行には、ターゲットの名前と、前提のファイルのハッシュ値を並べる。記録のファイルが壊れたり消えたりしたときは、すべてのターゲットを作り直せばよい。作り直しは遅いが、結果は正しい。記録を信じすぎて、作り直すべきものを作り直さないほうが、ずっと困った事態になる。

図 2-1 のように、記録のファイルはビルドの出力と同じ場所に置く。出力を消せば記録も消えるので、二つが食い違うことがない。

2.2.3　用語

この章で使った用語を、定義リストにまとめる。

ターゲット
: ビルドで作るファイル。



図 2.1　記録のファイルを置いたディレクトリの写真（見本）。

前提

　ターゲットを作るのに要るファイル。make の用語では prerequisite と呼ぶ。

ハッシュ値

　ファイルの中身から計算した、決まった長さの値。

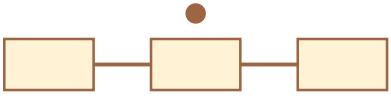


図 2.2　判定から作り直しまでの流れ。

　図 2-2 の「三つの箱を線でつないだ処理の流れ」のとおり、判定、順の決定、実行の三つの段階を経て、ターゲットが作られる。

　Internationalization and characterization of buildsystem interoperability requirements are discussed in the literature under names like reproducibility and incrementality. Unquestionably, well-understood, straightforward implementations outperform overcomplicated counterparts.

第 3 章　同時に動かす

3.1　動かす順

依存関係のグラフで、互いに依存しないターゲットは、同時に作ってよい。同時に動かす数を増やせば、多くのコアを持つマシンでビルドが速くなる。手順は次のとおりである。

1. 前提がすべてそろったターゲットを、待ちの列に入れる。
2. 動いている数が上限より少なければ、待ちの列から一つ取り出して動かす。
3. 一つ終わるたびに、そのターゲットを前提に持つターゲットを調べ、前提がそろったものを待ちの列に足す。
4. 待ちの列が空になり、動いているものもなくなれば終わる。

この手順では、待ちの列から取り出す順が、ビルドの時間を左右する[*1]。

3.2　出力を揃（そろ）える

3.2.1　混ざる出力

同時に動かすと、それぞれのコマンドの出力が混ざる。次の例は、二つのコンパイルを同時に動かしたときの出力である。

```
compiling app.c
compiling util.c
util.c:12: warning: unused variable 'count' in func
  tion 'parse_dependency_line_from_file'
app.c:40: warning: implicit conversion loses integ
  er precision: 'long' to 'int' in the expression of th
  e loop counter
done util.o
done app.o
```

警告の行が、どちらのコマンドのものかは、行の先頭のファイル名を見なければ分からない。出力が多いと、ファイル名のない行がどちらのものか分からなくなる。

*1 長くかかるターゲットを先に動かすと、全体の時間が短くなりやすい。最適な順を求める問題は難しいことが知られている（[1, pp. 12–14] の議論を参照）。

3.2.2　出力をためる

混ざらないようにするには、コマンドごとに出力をためておき、終わったときにまとめて書き出せばよい。ただし、長くかかるコマンドの出力は、終わるまで見えなくなる。進み具合を知りたい利用者のために、一行だけの進み具合の表示を別に出すのがよい。

```
run() {
  name=$1
  shift
  "$@" >"logs/$name.log" 2>&1
  code=$?
  cat "logs/$name.log"
  return $code
}
for t in app.o util.o; do
  run "$t" sh "rules/$t.sh" &
done
wait
```

3.3　失敗を扱う

3.3.1　止めるか続けるか

同時に動かしているコマンドの一つが失敗したとき、ほかのコマンドをどうするかを決めなければならない。既定では、新しいコマンドを動かさず、動いているものが終わるのを待ってから止まる。動いているものを途中で止めないのは、書きかけのファイルを残さないためである。`-k` のオプションを付けると、失敗したターゲットに依存しないものは作り続ける。大きなプロジェクトで、一度のビルドでなるべく多くの失敗を見つけたいときに便利である。

3.3.2　書きかけのファイル

コマンドが途中で失敗すると、ターゲットのファイルが書きかけのまま残ることがある。書きかけのファイルの時刻は前提より新しいので、時刻で判定するツールは、次のビルドでそれを作り直さない。これを避けるには、コマンドに一時的なファイルへ書かせ、成功したときにだけターゲットの名前に移せばよい。移す操作は、同じファイルシステムの中なら一度に行われるので、書きかけのターゲットが見えることはない。

```
tmp="$target.tmp.$$"
if sh "rules/$target.sh" >"$tmp"; then
  mv "$tmp" "$target"
else
  rm -f "$tmp"
  exit 1
fi
```

3.4　作ったものを使い回す

3.4.1　キャッシュ

同じ前提から作ったターゲットは、何度作っても同じ中身になるはずである。それなら、一度作ったものを取っておき、同じ前提のときはそれを使い回せばよい。前提のハッシュ値を並べたものから、さらにハッシュ値を計算し、それを名前にしてターゲットを保存しておく。次のビルドで同じ名前のものがあれば、コマンドを動かさずに写すだけで済む。

3.4.2　使い回せる条件

使い回しが正しいのは、コマンドが前提だけから結果を決める場合に限られる。コマンドが時刻や環境変数や乱数を読むと、同じ前提からでも違う結果になる。そうしたコマンドの結果を使い回すと、作り直したときと違うものが手元に残る。使い回しを安全にするには、コマンドが読むものをすべて前提に含めるか、読むものを固定するしかない。どちらも手間がかかるので、小さなツールでは、使い回しを明示的に選んだターゲットだけに限るのがよい。

3.5　まとめ

この本では、依存関係の読み込み、再ビルドの判定、同時に動かす制御の三つを作った。どれも短いスクリプトで書けるが、それぞれの判断には、時刻とハッシュ、順と同時実行のような、はっきりした選択がある[*2]。

参考文献

[1] Mokhov Andrey, Mitchell Neil, Peyton Jones Simon. Build Systems à la Carte. Proceedings of the ACM on Programming Languages, vol. 2. 2018.

[2] Stallman Richard M., McGrath Roland, Smith Paul D. GNU Make: A Program for Directing Recompilation. Free Software Foundation. 2023. https://www.gnu.org/software/make/manual/

[3] Knuth Donald E., Plass Michael F. Breaking Paragraphs into Lines. Software: Practice and Experience, vol. 11. 1981.

*2 段落の行分割にも、似た選択がある。[3] は、行ごとに決めるのではなく、段落の全体で最もよい分け方を探す方法を示した。ビルドツールの側の選択は [1, 2] が扱う。

索引

あ
依存関係
　グラフ　5

か
キャッシュ　15
　仕組み　15

さ
再ビルド
　時刻　9
　ハッシュ　9
時刻
　変化　10
失敗　14
出力　14

た
トポロジカルソート　5

は
ハッシュ
　記録　10
ビルドツール　5
並列　13 – 14

ま
make
　オプション　7
メモ化（キャッシュを見よ）